

MedGuard AI

Backend API Routing Reference

Django REST Framework · JWT Authentication · Microservice Architecture

Role: Backend Developer

HTTP method: ● GET · ● PATCH / PUT · ● POST · ● DELETE

1. Authentication & User Management

Base prefix: /api/server/auth/ | App: accounts | Auth required: **✗** (except logout & profile)

Method	Endpoint	Description	Auth
POST	/api/server/auth/register/	Register a new user account	Public
POST	/api/server/auth/login/	Login and receive JWT access + refresh tokens	Public
POST	/api/server/auth/token/refresh/	Refresh an expired access token (pass refresh token)	Public
POST	/api/server/auth/logout/	Blacklist refresh token / logout user	JWT
GET	/api/server/auth/profile/	Get currently authenticated user profile	JWT
PATCH	/api/server/auth/profile/update/	Update user name / password	JWT

2. Medicine Verification Module

Base prefix: /api/server/medicine/ | App: medicine | All routes require JWT

Method	Endpoint	Description	Auth
POST	/api/server/medicine/verify/text/	Verify medicine via text input (name/composition)	JWT
POST	/api/server/medicine/verify/image/	Verify medicine via uploaded image (triggers OCR)	JWT
POST	/api/server/medicine/verify/barcode/	Verify medicine via barcode value	JWT
GET	/api/server/medicine/result/<id>/	Retrieve a specific verification result by ID	JWT
GET	/api/server/medicine/search/	Search medicine by name (query param: ?q=)	JWT
GET	/api/server/medicine/<id>/	Get full details of a medicine record	JWT

3. OCR & Barcode Processing

Base prefix: /api/server/ocr/ and /api/server/barcode/ | App: ocr / barcode | All routes require JWT

Method	Endpoint	Description	Auth
POST	/api/server/ocr/extract/	Upload image; extract raw text via Google Vision OCR	JWT
POST	/api/server/ocr/parse-medicine/	Extract structured MFG / EXP / MRP fields from OCR text	JWT
POST	/api/server/barcode/decode/	Upload barcode image; decode using pyzbar	JWT
GET	/api/server/barcode/lookup/<barcode>/	Look up medicine record by decoded barcode value	JWT

4. RAG & FAISS Semantic Search

Base prefix: /api/server/rag/ | App: agents (or medicine) | All routes require JWT

Method	Endpoint	Description	Auth
POST	/api/server/rag/search/	Perform FAISS similarity search; returns top-K matches + similarity scores	JWT
GET	/api/server/rag/index/status/	Check FAISS index health (record count, dimension, status)	JWT
POST	/api/server/rag/index/rebuild/	Trigger re-indexing of FAISS from latest medicine dataset (Admin only)	JWT

5. Symptom Remedy LLM Module

Base prefix: /api/server/symptom/ | App: symptom | All routes require JWT

Method	Endpoint	Description	Auth
POST	/api/server/symptom/remedy/	Submit symptom text; receive safe home remedy from Ollama LLM	JWT
GET	/api/server/symptom/history/	Get current user's symptom query history (if logging enabled)	JWT
DELETE	/api/server/symptom/history/clear/	Clear all symptom history for current user	JWT
GET	/api/server/symptom/llm/status/	Check Ollama LLM service health and availability	JWT

6. Verification History

Base prefix: /api/server/history/ | App: history | All routes require JWT

Method	Endpoint	Description	Auth
GET	/api/server/history/	List all verification history entries for current user	JWT
GET	/api/server/history/<id>/	Retrieve a single history entry with full details	JWT
DELETE	/api/server/history/<id>/delete/	Delete a specific history entry	JWT
DELETE	/api/server/history/clear/	Clear all history for the current user	JWT
GET	/api/server/history/stats/	Get summary statistics: counts by result status	JWT

7. File Upload Service

Base prefix: /api/server/files/ | App: files | All routes require JWT

Method	Endpoint	Description	Auth
POST	/api/server/files/upload/	Upload medicine image or barcode image; returns file URL	JWT
GET	/api/server/files/<id>/	Retrieve uploaded file metadata by file ID	JWT
DELETE	/api/server/files/<id>/delete/	Delete an uploaded file	JWT

8. Core & Health Check

Base prefix: /api/server/core/ | App: core | No auth required for health endpoints

Method	Endpoint	Description	Auth
GET	/api/server/core/health/	Overall system health check (all services)	Public
GET	/api/server/core/health/db/	PostgreSQL database connectivity check	Public
GET	/api/server/core/health/faiss/	FAISS vector index status check	Public
GET	/api/server/core/health/llm/	Ollama LLM service status check	Public
GET	/api/server/core/version/	Return current API version and build info	Public

9. Django urls.py Structure

How all routes wire together across medguard/urls.py and each app's urls.py

medguard/urls.py (Root URL Config)

```
# medguard/urls.py from django.urls import path, include urlpatterns = [
path('api/server/auth/', include('apps.accounts.urls')),
path('api/server/medicine/', include('apps.medicine.urls')),
path('api/server/ocr/', include('apps.ocr.urls')),
path('api/server/barcode/', include('apps.barcode.urls')),
path('api/server/rag/', include('apps.agents.urls')),
path('api/server/symptom/', include('apps.symptom.urls')),
path('api/server/history/', include('apps.history.urls')),
path('api/server/files/', include('apps.files.urls')),
path('api/server/core/', include('apps.core.urls')), ]
```

apps/symptom/urls.py (Example App-Level Routing)

```
# apps/symptom/urls.py from django.urls import path from . import views urlpatterns = [
path('remedy/', views.SymptomRemedyView.as_view(), name='symptom-remedy'),
path('history/', views.SymptomHistoryView.as_view(), name='symptom-history'),
path('history/clear/', views.ClearHistoryView.as_view(), name='symptom-clear'),
path('llm/status/', views.LLMStatusView.as_view(), name='llm-status'), ]
```

10. Request / Response Conventions

Standard Success Response

```
{  "status": "success",  "message": "Medicine verified successfully.",  "data": { ... },
"timestamp": "2025-03-16T10:30:00Z" }
```

Standard Error Response

```
{  "status": "error",  "message": "Medicine not found in database.",  "error_code":
"MEDICINE_NOT_FOUND",  "timestamp": "2025-03-16T10:30:01Z" }
```

HTTP Status Codes Used

Method	Endpoint	Description	Auth
GET	200 OK	Successful data retrieval	Public
POST	201 Created	Resource created successfully	Public
POST	200 OK	Action performed (verify, search, remedy)	Public
GET	400 Bad Request	Validation error / malformed input	Public
GET	401 Unauthorized	JWT missing or invalid	Public
GET	403 Forbidden	Authenticated but insufficient permissions	Public
GET	404 Not Found	Resource does not exist	Public
GET	500 Internal Server Error	Unhandled server / service error	Public

11. JWT Authentication Header

All protected endpoints require the following header:

```
Authorization: Bearer <access_token>
```

Tokens are obtained from POST /api/server/auth/login/ and refreshed via POST /api/server/auth/token/refresh/.